

Unit 1 · Topic Notes

Introduction to Git & GitHub — Branches, Pull Requests, Issues & Project Boards

Course: Full Stack Application Development with MERN **Prerequisite:** Basic `git init / add / commit`

Learning Objectives

By the end of this topic, you should be able to:

1. Create and manage branches for isolated feature work
2. Open, review, and merge a Pull Request (PR) using GitHub's workflow
3. Log, label, and track bugs/features using GitHub Issues
4. Set up and use a GitHub Project (Kanban) board to run a sprint
5. Simulate a real team workflow on a shared MERN repo

1. Why This Matters

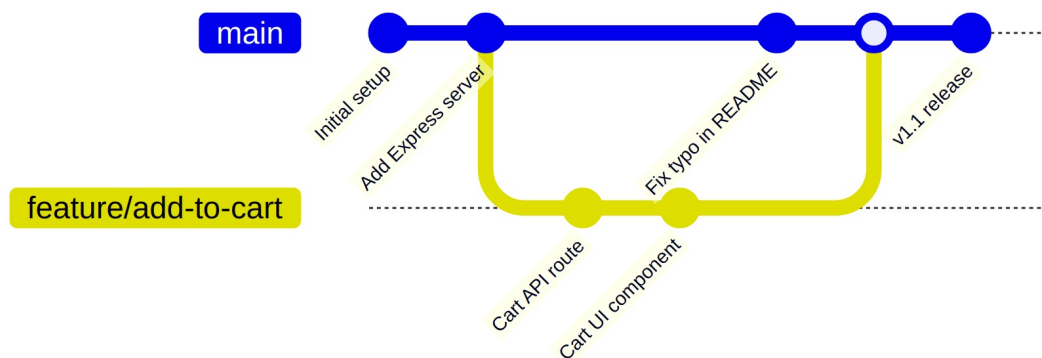
Imagine three people edit `server.js` at the same time on `main`. What happens? Chaos — overwritten code, broken builds, no history of *why* something changed.

Branches + PRs + Issues + Boards are how real engineering teams (Amazon, Razorpay, Zomato, every startup you'll intern at) avoid exactly this. This is not “extra GitHub trivia” — it **is** the industry workflow.

2. Branching

2.1 Core idea

A branch is a movable pointer to a commit. `main` should always be deployable; all new work happens on a separate branch and only joins `main` after review.



Git branching diagram

2.2 Naming Convention

Prefix	Use case	Example
<code>feature/</code>	New functionality	<code>feature/user-auth</code>
<code>fix/</code> or <code>bugfix/</code>	Bug fix	<code>fix/cart-total-nan</code>
<code>chore/</code>	Config, deps, tooling	<code>chore/update-eslint</code>
<code>hotfix/</code>	Urgent prod fix	<code>hotfix/payment-crash</code>

2.3 Commands

```
# Create and switch to a new branch off main
git checkout -b feature/add-to-cart

# (modern alias — Git 2.23+)
git switch -c feature/add-to-cart

# See all branches
git branch -a

# Push the branch to GitHub the first time
git push -u origin feature/add-to-cart

# Switch back to main
git switch main

# Delete a branch after it's merged
git branch -d feature/add-to-cart
```

2.4 Branching Strategy for Student Projects

For a semester-length MERN capstone, full Gitflow is too heavy for a 2–4 person team. Use the **GitHub Flow** simplification instead:

`main` (always deployable) → short-lived `feature/*` branches → PR → merge → delete branch.

Gitflow (`develop` , `release/*` , `hotfix/*`) is what larger teams with scheduled releases use — good to know about, but not something you'll need for this course's projects.

2.5 Merge Conflicts

```
# On feature branch, someone edited line 12 of App.js
# On main, someone else also edited line 12
git switch main
git pull
git switch feature/add-to-cart
git merge main
# CONFLICT (content): Merge conflict in src/App.js
```

Open the file and look for the conflict markers:

```
<<<<<< HEAD
const TAX_RATE = 0.18;
=====
const TAX_RATE = 0.12;
>>>>>> main
```

Resolve → `git add App.js` → `git commit` → conflict resolved. This is the #1 real-world skill gap for junior developers, so it's worth practicing until it feels natural.

3. Pull Requests (PRs)

3.1 What & Why

A PR is a request to merge one branch into another, with a review step in between. It is GitHub's mechanism for code review, discussion, and CI checks — **never push directly to `main`** on team projects; use branch protection rules to enforce this.



Pull request workflow diagram

3.2 Opening a Good PR

A PR description template (save as `.github/pull_request_template.md` in the repo):

```
## What does this PR do?
Adds "Add to Cart" button and connects it to POST /api/cart

## Related Issue
Closes #14

## How to test
1. Run `npm run dev`
2. Go to /products, click "Add to Cart"
3. Confirm item appears in /cart

## Screenshots
(attach before/after if UI changed)

## Checklist
- [ ] Code follows project style guide
- [ ] Tested locally
- [ ] No console.log left in code
```

The `Closes #14` keyword is doing real work: when this PR is merged, GitHub **automatically closes Issue #14** — a small detail that saves a lot of manual bookkeeping.

3.3 Merge Strategies

Strategy	History result	When to use
Merge commit	Keeps every individual commit + adds a merge commit	Default; preserves full history
Squash and merge	All branch commits collapse into one commit on <code>main</code>	Recommended for student teams — keeps <code>main</code> history clean
Rebase and merge	Branch commits replayed individually, no merge commit	Intermediate/advanced teams who want linear history

Recommendation for this course: Squash and merge. It hides “wip”, “fix typo”, “asdf” commit messages from `main` and keeps history readable for grading.

3.4 Code Review Etiquette

- Review within 24 hours — don’t block teammates
- Comment on the *code*, not the *person* (“this loop re-renders on every keystroke” not “why did you write this”)
- Approve, Request Changes, or Comment — use the right one
- Small PRs (< 400 lines) get reviewed properly; giant PRs get rubber-stamped

4. Issues

4.1 Anatomy of a Good Issue

```
Title: Cart total shows NaN when quantity is 0

**Description**
When a product's quantity is set to 0 in the cart, the total price
displays "NaN" instead of "Rs. 0".

**Steps to reproduce**
1. Add any product to cart
2. Set quantity to 0
3. Observe total

**Expected:** Total shows Rs. 0
**Actual:** Total shows NaN

**Environment:** Chrome 128, localhost
```

4.2 Labels, Assignees, Milestones

Label	Meaning
bug	Something is broken
enhancement	New feature request
good first issue	Beginner-friendly, small scope
help wanted	Team needs extra hands
documentation	README/docs only

Assign an **assignee** (who's doing it) and a **milestone** (e.g., "Sprint 2 — Auth & Cart") so issues roll up into a deadline.

4.3 Issue Templates

`.github/ISSUE_TEMPLATE/bug_report.yml` :

```
name: Bug Report
description: Report something broken
labels: ["bug"]
body:
  - type: textarea
    id: what-happened
    attributes:
      label: What happened?
      placeholder: Tell us what went wrong
    validations:
      required: true
  - type: input
    id: expected
    attributes:
      label: Expected behavior
    validations:
      required: true
```

This forces every bug report in the class repo to follow the same structure — great for grading consistency too.

4.4 Linking Issues Everywhere

- In commit messages: `git commit -m "fix cart NaN bug, refs #14"`
- In a PR description: `Closes #14` , `Fixes #14` , `Resolves #14` (all auto-close on merge)

5. Project Boards (GitHub Projects)

5.1 What It Is

GitHub Projects is a Kanban/table board built directly on top of your Issues and PRs — no separate tool needed (no Trello/Jira required for a student project). Each card *is* an Issue or PR; dragging it across columns doesn't just move a sticky note, it updates a real field (`Status`) on that Issue.



Project board flow diagram

5.2 Setting One Up for a Capstone Project

1. Repo → **Projects** tab → **New project** → choose **Board** view
2. Add columns: `Backlog` , `To Do` , `In Progress` , `In Review` , `Done`
3. **Automation (built-in workflows):**
 - New issue → auto-added to `Backlog`
 - PR opened → linked issue auto-moves to `In Review`
 - Issue closed → card auto-moves to `Done`
4. Add **custom fields**: `Priority` (P0/P1/P2), `Sprint` , `Story Points`
5. Switch to **Table view** to sort/filter — useful for seeing who did what

5.3 Why This Matters for Teamwork

Your project board is your proof of work — instructors look at who opened issues, who moved cards, and whose PRs actually got merged, not just who talked the most in the group chat.

6. End-to-End Walkthrough

Scenario: 4-person team building a MERN e-commerce app. Feature: “Add to Cart.”

1. Teammate A opens **Issue #14**: “Add to Cart button not connected to backend” → labels `enhancement` , assigns self, adds to `Sprint 1` milestone → board auto-adds card to `Backlog` , A drags it to `To Do`
2. A creates branch: `git switch -c feature/add-to-cart`
3. A builds the POST `/api/cart` route + `<AddToCartButton />` , commits, pushes
4. A opens a PR: title `Add to Cart button + API` , description uses the template, `Closes #14` → board auto-moves the card to `In Review`
5. Teammate B reviews: leaves 2 comments, requests changes → A pushes a fix commit → B approves
6. A **squash and merges** → `main` gets one clean commit → branch auto-deleted → Issue #14 auto-closes → board card auto-moves to `Done`
7. Repeat for the next issue — this loop *is* the semester

7. Hands-On Lab Activity (30-40 min, pairs or teams of 3-4)

1. One student creates a shared repo, adds teammates as collaborators
2. Add branch protection on `main` : require PR before merging, require 1 approval
3. Each teammate opens 1 issue (mix of `bug` and `enhancement`)
4. Set up a Project board with the 5 columns from §5.2
5. Each teammate: branch → tiny change (e.g., add their name to a `CONTRIBUTORS.md`) → PR → get reviewed by a teammate → merge
6. End state check: board shows all cards in `Done` , `main` has a clean squashed history, all issues closed

8. Command Cheat Sheet

Task	Command
New branch	<code>git switch -c feature/xyz</code>

Push new branch	<code>git push -u origin feature/xyz</code>
List branches	<code>git branch -a</code>
Merge main into your branch (before PR)	<code>git merge main</code>
Delete local branch	<code>git branch -d feature/xyz</code>
Delete remote branch	<code>git push origin --delete feature/xyz</code>
See PR from CLI (needs GitHub CLI)	<code>gh pr create</code> / <code>gh pr list</code> / <code>gh pr checkout 14</code>
Close an issue via commit	<code>git commit -m "fix: cart total, closes #14"</code>

9. Quick Assessment

1. Why should `main` never be committed to directly on a team project?
2. What's the difference between "Merge commit" and "Squash and merge"?
3. Write the PR description keyword that auto-closes Issue #22 on merge.
4. What are conflict markers (`<<<<<<<` , `=====` , `>>>>>>>`) telling you?
5. On a GitHub Project board, what actually happens when you drag a card from `In Progress` to `Done` ?

10. Further Reading (Optional)

- GitHub Docs — "About pull requests" (docs.github.com)
- GitHub Docs — "About issues" (docs.github.com)
- GitHub Docs — "About GitHub Projects" (docs.github.com)
- Atlassian's Git branching model comparisons (for the Gitflow vs GitHub Flow mention in §2.4)

Notes prepared for Full Stack Learning — MERN course, Unit 1.